

沈阳航空航天大学
SHENYANG AEROSPACE UNIVERSITY

生产实习总结报告

学 院 人工智能学院
专 业 物联网工程
班 级 物联网 2202
学 号 223428010210
姓 名 陈梓欣
指导教师 肖俊峰
日 期 2025 年 12 月 31 日

沈阳航空航天大学人工智能学院

沈阳航空航天大学人工智能学院

一、课程：物联网工程 2022 级生产实习

二、时间：自 2025 年 12 月 22 日起至 2025 年 12 月 31 日

三、实习内容及安排

培训内容	学时	主题	课程内容概述
第 1 天	6	开营	1. 参观东软，了解企业文化 2. 开营仪式，了解未来需求 3. 行业讲座
第 2 天	6	项目设计方案	1. 嵌入式项目设计流程 2. 嵌入式软件系统架构 3. 嵌入式硬件组成结构设计 4. 环境搭建使用 5. 控制器的基本使用 6. API 接口应用 7. 控制器硬件结构
第 3 天	6	微控制器	1. 接口使用操作演示 2. 内部资源应用流程 3. STM32 CMSIS 4. 温湿度传感器数据采集操作方式 5. 温湿度传感器数据采集时序逻辑分析
第 4 天	6	串口设备驱动	1. 串口设备介绍 2. 串口设备原理图讲解 3. 串口设备发送数据代码 4. 串口设备接收数据代码
第 5 天	6	温湿度传感器	1. 单总线协议 2. 温湿度传感器数据解析方式 3. 温湿度传感器数据传输协议讲解
第 6 天	6	Qt 编程	1. Qt 安装 2. Qt 项目简介 3. Qt 窗体文件的构成 4. 事件、信号与槽 5. 界面布局 6. 对话框 7. 主窗口
第 7 天	6	Qt 编程	1. 常用界面设计组件的使用 2. 基本绘图系统
第 8 天	6	Qt 编程	1. 图表简介 2. 折线图 3. 曲线图 4. 柱状图

			5. 饼图
第 9 天	6	Qt 编程	1. 数据库设计规范 2. 数据库操作方式 3. Qt 操作 SQLite 数据库 4. 使用表格或图表展示温度数据
第 10 天	6	测试、答辩	1. 项目整体测试 2. 项目答辩 3. 项目评审 4. 项目成果展示

实习评语：

实习纪律(满分 10)	
岗位表现(满分 70)	
总结报告(满分 20)	
实习成绩	

指导教师签字：

学生签字：陈辉欣 日期：2025 年 12 月 31 日

生产实习总结：（3000 字 实习基本情况、完成实习内容情况、参加实习单位实际工作情况、遵守实习队纪律和实习单位规章制度情况、主要收获和存在的问题等）

本次生产实习在东软集团进行，实习周期为十个工作日。东软集团作为国内软件与信息技术服务领域的领军企业，拥有成熟的项目管理体系、规范化的研发流程以及多方向的技术沉淀，为本次实习提供了优质的工程实践平台与资源支持。本次实习采用线下办公与线上协作结合的方式展开，使技术资料共享、远程调试支持与团队协作沟通更加高效灵活，也让实习生提前适应了企业混合式开发模式下的真实工作节奏与协同方式。

本次实习的核心目标是熟悉企业级项目开发流程、提升 Qt 项目开发能力，并掌握串口通信在可视化监控系统中的实际应用方法。在任务设计上，以温湿度监测系统作为软硬件结合实践载体，覆盖了从硬件传感器接入、下位机固件开发、串口通信协议设计、数据上报实现、上位机数据解析、实时可视化展示、异常处理与联调整体调试的完整流程。整个实习过程强调工程化思维训练、代码规范实践、通信可靠性保障、UI 实时渲染稳定性验证、以及企业协作模式下的任务交付能力培养。

在实习初期，我们首先进入硬件联调与固件烧录验证阶段。DHT11 传感器作为系统感知核心器件，通过单总线协议与 STM32 主控芯片的 PA15 引脚进行数据通信。为确保传感器总线信号稳定可靠，我们使用万用表对传感器供电电压、上拉电阻焊接状态与阻值合理性、GPIO 电平输出能力、以及串口交叉收发连接链路进行了快速核验，避免由于电源不稳、总线电平异常或串口线序错误导致的通信失败。随后使用 Keil 在线烧录固件并进入运行监测，观察 LED 是否保持稳定周期翻转作为系统心跳指示，并通过串口调试助手（如 SSCOM）实时监视 USART1 输出的数据帧是否持续上报、刷新节奏是否符合预期、数据帧是否完整稳定且无乱码或发送中断。此外，还增加了串口调试助手检测数据传输效果这一要点，连续监测固件运行下的温湿度数据传输表现，确保通信链路在硬件层面无阻塞、数据帧格式一致、刷新周期稳定可控。通过以上验证，确认系统成功进入主循环采集与上报状态，为后续的软件解析与可视化联调奠定了坚实基础。

在下位机固件开发阶段，我们以周期采集+字符串打包+串口上报的方式完成温湿度数据上报模块设计。固件通过 DHT11 读取温湿度值后，调用 Serial_Printf 函数对温湿度字段进行格式化打包，生成统一规范的数据字符串，并调用串口发送模块将数据帧写入 USART1 发送寄存器，完成数据上报。固件主循环结构简洁可靠、通信逻辑明确，避免复杂的缓冲与中断堆叠风险，符合企业嵌入式通信工程中“可靠优先、逻辑简洁、发送稳定、格式统一、调度清晰”

的设计原则。我们通过 Keil 仿真断点调试检查 DHT11 初始化与读取流程、观察字符串打包是否符合格式要求、并通过在线运行验证固件逻辑无阻塞、串口输出持续稳定，证明固件通信模块在工程层面满足数据上报可靠性要求。

上位机软件开发阶段基于 Qt 5 的 Widgets + SerialPort 模块完成数据接收、解析与展示。软件运行后正确枚举串口端口，并支持串口的打开与关闭控制。通过串口接收回调函数 `readyRead` 读取串口缓冲区上报的数据帧，并通过字符串拆分+数值转换提取温湿度字段，更新至 UI 状态标签与数据表格显示。同时基于 Qt Charts 实现温湿度曲线的实时增长绘制，验证曲线渲染是否连贯、坐标刻度是否稳定适配、数据滚动增长是否符合预期、界面重绘是否及时响应。在软件调试阶段，我们使用串口助手模拟发送标准格式温湿度字符串，验证数据接收是否完整稳定、解析字段是否正确、界面更新是否及时响应。此外我们还模拟传感器读取失败、发送异常数据帧等场景，验证上位机是否能正确识别错误并更新提示状态，而不出现崩溃或界面卡死现象。通过这些调试验证，确认软件通信链路完整、解析逻辑准确、UI 刷新稳定、异常可被捕获、主程序不中断运行，表明系统具备基础的通信鲁棒性与可视化稳定性。

在实习过程中，我参与了实习单位真实开发环境下的项目协作与任务交付流程。在需求分析阶段，与团队共同讨论系统功能目标、通信格式要求、数据刷新节奏与异常提示逻辑，提前对系统整体运行周期与通信帧格式进行了统一规范定义。在功能设计阶段，我按照公司代码工程模块化与串口通信可靠性规范完成模块划分，明确了传感器采集、字符串打包、串口上报、Qt 串口接收、数据解析、UI 刷新、异常处理等模块的功能边界，确保团队开发方向一致。在代码实现阶段，我遵循企业编码规范完成逻辑实现、编写关键函数注释、保证代码结构清晰可读，便于后续维护与联调协作。在联调阶段，我参与了硬件固件+上位机串口通信联调与可视化验证工作，使用万用表、Keil 在线运行调试、串口助手、日志输出与 UI 渲染监测等方法进行验证，确保系统稳定进入主循环运行。

实习期间，我始终严格遵守实习队纪律要求与实习单位的规章制度。坚持做到准时上下班、按时参加团队会议、及时汇报工作进展、严格使用公司提供的开发环境与工具、不随意占用通信端口、不私自更改项目资源、不泄露开发数据与调试信息。在工作态度上保持积极主动、认真负责、任务优先、响应及时、交付可靠，体现了企业工程实践中的基本职业素养。

本次实习的主要收获包括：第一，掌握了 Qt 5 串口模块在实际工程监控系统中的稳定使用方法，包括串口枚举、打开/关闭、接收回调、缓冲读取、解析刷新等工程实践能力；第二，掌握了 STM32 固件通信模块的设计与验证方法，理解了嵌入式通信工程中“格式统一、发送

稳定、调度可靠、总线电平可控、通信节奏稳定”的设计原则；第三，积累了串口通信软硬件联调工程实践经验，理解了硬件电源、GPIO 电平、总线信号、串口线序、数据帧格式、UI 刷新节奏与异常提示逻辑的联动关系；第四，提升了团队协作与模块联调沟通能力，熟悉了企业协作模式下的任务分配、联调调试、异常定位与功能验证工作流程；第五，强化了工程项目交付意识与代码规范实践能力，学会了在实际开发中兼顾可靠性与界面响应稳定性的系统设计方法。

在实习中也存在一些问题与不足：第一，初期在串口通信联调节奏与 UI 实时刷新匹配上耗费了较多调试时间，说明在通信节奏控制与上位机重绘响应机制上仍需积累更多实践经验；第二，当前系统在高频数据刷新场景下图表实时渲染仍存在轻微延迟，未来可进一步优化绘图线程分离与重绘触发逻辑；第三，在串口异常帧输入与读取失败提示的容错策略上仍有优化空间，未来可引入更规范的协议帧头校验或 JSON 上报结构以提高识别准确度；第四，在数据上报模块扩展与后期存储分析模块设计方面，尚未接入数据库记录模块，未来可进一步对接 MySQL 存储与连接池管理以支撑更高并发与更稳定的长期运行场景。

此外，本次实习还让我对企业工程项目中的可靠性设计理念与系统思维方法有了更深层的理解。与学校课程中偏向功能实现的开发模式不同，企业项目更强调稳定运行、异常兜底、接口规范、跨模块协同、以及可长期维护的架构设计。在串口通信与数据上报的联调过程中，我逐渐意识到通信并不是单一的“发送与接收”，而是包含信号质量、电源稳定、周期匹配、协议格式、UI 响应触发、资源占用冲突、以及错误状态反馈链路在内的整体系统工程。本次项目虽然功能模块相对简洁，但在企业化的开发流程下，我们依然对每个关键节点进行了联通验证与状态确认，这种从“能用”到“可靠”的调试思路转变，是我本次实习最大的认知提升之一。同时，我也意识到一个完整的可视化监控系统未来仍可进一步向结构化协议上报、数据库存储分析、连接池管理、物联网平台对接、以及更高频通信下的图形渲染优化方向演进。

总体来看，本次生产实习通过规范化的软硬件联调流程、Keil 固件运行验证、Qt 串口可视化监控实践、通信稳定性测试与异常处理联调验证，完成了温湿度监测系统的全流程开发闭环。项目实现目标清晰、通信链路可靠、UI 刷新稳定、异常处理基础容错有效、模块划分规范合理，满足了实习任务与企业工程实践的基础要求。此次实习不仅让我深入了解了企业项目开发流程，也让我在实际通信工程联调与 UI 可视化监控系统实践中积累了工程化经验，对未来从事物联网数据可视化、串口通信联调与 Qt 监控系统开发方向具有重要意义。

1 引言

温湿度采集与监测系统项目设计与实施方案总结

1.1 项目概述

温湿度实时监测系统项目是一个基于 STM32 嵌入式开发平台与 Qt 6 可视化框架的综合性软硬件解决方案,旨在实现对环境温度与湿度数据的精准采集、稳定传输、实时显示及历史统计分析。系统围绕物联网数据采集与串口可视化监控需求展开设计,具备高可靠性、低延迟传输、动态趋势呈现及多维度数据统计能力,可满足日常环境监测、工业辅助检测、数据记录分析等应用场景。

在下位机硬件设计方面,系统以 STM32 微控制器为核心控制单元,协同 DHT11 温湿度传感器完成环境数据的周期性采集,并通过内置 UART 串口通信模块将数据编码为结构化 ASCII 文本上传至上位机。为了保证传感器通信稳定性,系统在 DHT11 读取时加入延时与错误重试机制,避免因采样频率过高导致的采集失败,从而提升整体系统的数据可信度与抗干扰能力。

上位机软件基于 Qt 5 框开发,采用 QSerialPort 串口模块完成数据接收与指令交互,并结合 QtCharts 折线图组件实现温湿度数据的动态趋势曲线绘制。系统支持串口端口选择、波特率输入及通信参数自定义,用户可根据不同设备和传输需求灵活调整通信配置。接收端解析串口数据后,系统自动更新 GUI 界面,包括当前温度、当前湿度数值显示,并实时参与历史数据统计计算,维护历史最高温度、最低温度、最高湿度、最低湿度等关键指标,增强系统的环境监测感知能力。此外,系统设计本地缓存数据队列用于图表追加绘制,并限制数据点数量以防内存膨胀,确保长期运行的性能稳定。

整体系统构建了嵌入式实时采集 → 串口稳定传输 → 上位机解析展示 → 数据趋势可视化的完整数据链路,用户可通过直观界面了解当前环境状态和历史极值变化,为温湿度监测提供了高效、便捷且可视化的数据管理方案,是一个兼具实用性、扩展性和工程应用价值的软硬件融合项目。

1.2 需求分析

本系统旨在实现环境温湿度数据的稳定采集、可靠上报与实时可视化监控,覆盖从传感器硬件感知到串口通信链路再到上位机图形展示的完整流程。下位机

以 STM32 MCU 与 DHT11 传感器作为核心硬件组合，采用单总线协议进行温湿度数据读取。为提升采集可靠性，系统在固件层加入了周期性延时调度、读取失败重试与错误状态反馈机制，以降低因传感器响应抖动、外界干扰或时序偏差导致的数据读取失败概率，从源头保证数据输出的稳定性和有效性。温湿度采集结果并非以裸数据直接发送，而是经过固件层进行统一的字符串封装与格式规范处理，生成结构化的 ASCII/ASCII 兼容文本数据帧，并通过 USB-TTL 适配器建立完整的 UART 物理通信链路，借助 USART1（PA9-TX / PA10-RX）串口端口完成数据上报。通信格式采用易解析、低冗余、抗乱码的统一字段顺序与稳定分隔符设计，确保上位机在不同采集周期、不同数据变化下仍可保持一致的解析逻辑，避免数据错位、字段缺失、粘包或读取中断等问题。系统要求串口通信具备低延迟、抗干扰、无明显丢包、无乱码、可持续上报的能力特征，确保系统在长时间运行下数据链路仍保持连贯，为上位机提供可靠数据源输入。

上位机软件基于 Qt 5 Widgets 框架进行 GUI 构建，核心通信模块采用 QSerialPort 进行串口枚举、连接控制与数据接收回调，绘图模块使用 QtCharts 进行折线图可视化渲染。系统需支持串口端口可选择、波特率可自定义输入、串口可打开/关闭、数据可清空、数据接收不中断等基本串口监控能力，确保通信控制权可被用户灵活管理。上位机不仅仅是“显示数据”，还需具备实时状态更新、历史极值统计、曲线趋势绘制与性能限制保护等能力。温湿度数据显示分为两类：一类是当前最新采集值，要求无延迟刷新至 UI 状态标签；另一类是历史最高与最低温湿度，系统需对每条新数据进行对比更新与界面刷新，形成运行期间自动统计极值的能力，避免依赖外部存储或人工计算。数据可视化曲线要求图表具备坐标轴范围合理适配、曲线实时更新、数据点动态绘制、长时间运行不卡顿等特征。为保障性能稳定性，系统采用本地缓存队列管理采集数据，并限制图表最大数据点数量，通过数据滚动机制避免内存无限增长和图表渲染堆叠造成性能下降，确保系统在长期运行下仍可保持 GUI 流畅、无闪烁、无崩溃、可持续重绘。此外，系统在软件层预留了数据库接口，用于未来扩展历史数据存储、查询、生命周期统计、趋势分析与数据管理能力，使系统架构不仅满足当前需求，还具备良好的可演进与可维护工程化基础。

综上，本系统需求强调采集可靠、通信稳定、解析一致、可视化实时、性能

可控、结构可维护、未来可拓展等核心设计目标，以工程化闭环通信与可视化监控作为主要能力导向，兼顾软硬件协同稳定运行与用户操作灵活性。该系统既满足当前环境温湿度实时监控需求，也为后续升级工业物联网监测、多传感器上报、数据库管理与高频通信可视化渲染方向提供架构铺垫。

1.3 运行环境

硬件环境：本项目开发运行在一台基于 Microsoft Windows 10 的桌面/笔记本主机上，其配置如下：

处理器：Intel Core i5/i7 系列（x64 架构，支持多线程编译与串口高频数据处理），主频满足 Qt 实时 GUI 渲染与串口数据接收任务的稳定运行

系统内存：8GB - 32GB（本机建议≥16GB），用于支撑 Qt 应用、图表绘制、本地缓存队列及串口通信缓冲

硬盘：具备至少 2GB 可用空间，用于 Keil、Qt Creator、驱动与项目构建产物存放

USB 端口：至少 1 个，用于连接 CH340 USB-TTL 适配器进行串口通信。

系统类型：64 位操作系统，基于 x64 的处理器架构

嵌入式硬件环境：

核心硬件选型如表 1.1 所示。

表 1.1 核心硬件选型

硬件模块	型号/规格	功能点详细内容
微控制器	STM32F103C8T6 （主流经济型）	核心控制单元，驱动 DHT11、封装数据、控制串口传输。
温湿度传感器	DHT11	单总线数字传感器，采集环境温度（0-50℃，精度±2℃）、湿度（20%-90%RH，精度±5%RH）。
串口模块	CH340 (USB 转 TTL)	实现 STM32 与电脑的 USB 串口通信，提供 5V/3.3V 供电。
辅助硬件	杜邦线、3V3 电源	电路搭建与供电支持。

硬件电路连接：

i. ST-LINK 调试器（程序下载与 SWD 调试接口）

用于烧录固件、在线调试、时钟与数据同步，连接如表 1.2 所示：

表 1.2 ST-LINK 引脚连接说明

ST-LINK 引脚	连接对象	STM32 引脚/说明
SWCLK	STM32 GND	SWD 时钟信号。
SWDIO	DHT11	SWD 调试数据 I/O。
GND	STM32 RXD	共地。
3.3V	STM32 3V3	为目标板提供 3.3V 参考/供电（可选）

该接口使用 SWD（Serial Wire Debug）协议，相比 JTAG 连接更少引脚、更快、更稳定，适用于 STM32 开发调试。

- ii. USB 转 TTL（基于 CH340/CH340G/CH340C/CH340N 的串口通信链路）

系统使用 USB-TTL 模块实现下位机与上位机的数据通信，连接方式如表 1.3 所示：

表 1.3 USB-TTL 引脚连接说明

USB-TTL 引脚	连接对象	STM32 引脚/说明
GND	STM32 GND	共地，保证串口信号参考电平一致。
RXD	DHT11	PA9（USART1_TX），USB-TTL 接收 MCU 发送的数据。
TXD	STM32 RXD	PA10（USART1_RX），MCU 接收上位机串口发送的数据。

该连接构成完整 UART 通信链路，部分通信参数（波特率、串口）将在上位机 Qt 软件中支持自定义配置。

- iii. DHT11 传感器（温湿度采集，单总线协议）

系统使用 DHT11 进行环境温湿度采集，采用单总线通信方式连接如表 1.4 所示：

表 1.4 DHT11 引脚连接说明

DHT11 引脚	连接对象	STM32 引脚/说明
VCC	3.3V 电源	由 STM32 3V3 供电（也支持 5V 兼容供电）。
DATA	温湿度数据信号线	PA15（GPIO 口），用于单总线数据采集。
GND	STM32 GND	传感器与 MCU 共地。

该连接基于单总线协议实现温湿度数据采集链路，通过 GPIO 读取传感器数字信号并配合上拉电阻提升抗干扰能力，为下位机提供可靠的数据源输入。

软件环境：项目的开发和运行依赖以下软件及工具：

操作系统：Microsoft Windows 10（兼容 Keil 5、Qt Creator 与 CH340 串口驱动）。

编译器：MinGW（64-bit 版本，支持 C++11 标准），用于 QT 项目的编译

与构建。

开发工具：下位机开发工具为 Keil MDK 5 (Keil 5)，用于 STM32 固件工程开发、SWD 下载与在线运行调试；上位机开发工具为 Qt 6 / Qt Creator，使用 QSerialPort 模块完成串口端口枚举、打开/关闭与数据接收回调，使用 QtCharts 进行温湿度折线图的实时绘制与坐标轴适配；USB-TTL 适配器采用 CH340/CH340G/CH340N/CH340C 驱动，用于构建 STM32 与电脑的 USB-TTL 串口通信链路，提供稳定的虚拟串口端口支持。

其他辅助工具：串口辅助调试工具包括 SSCOM / SSCOM 串口助手 / SSCOM 32 / SSCOM5 / SSCOM42 / SSCOM V5.13 / SSCOM V5.12 / SSCOM V5.11 / SSCOM V5.10 等串口监控软件，用于验证下位机数据帧持续上报、无乱码、无丢帧、无丢包、字段顺序一致；系统绘图调试不涉及高频医学信号场景，但采用了数据点数量限制与本地缓存队列机制，避免 UI 长期运行下出现渲染堆叠或内存增长风险

网络环境：项目运行在本机本地通信模式下，不涉及远程云服务器部署。网络模块未被启用，数据链路以串口为主，无需 TCP/IP 或局域网支持即可完成通信测试与可视化渲染。

数据库环境：项目当前版本不涉及数据库连接、存储与查询操作，因此不依赖 MySQL、Navicat 或 Navicat Premium 等数据库管理工具，也未启用数据库连接池、端口、用户名或密码配置。系统架构虽预留未来数据库扩展接口，但本次实习/开发过程未涉及数据库读写或大数据存储场景，无数据库敏感信息参与项目运行。

虚拟串口工具：本项目未使用虚拟串口对进行数据模拟，数据通信直接通过 CH340 USB-TTL 模块 连接 MCU (PA9/PA10 - USART1) 与电脑的真实 USB-串口端口完成链路构建与验证，确保端口枚举稳定、通信无冲突、信号参考一致。

2 下位机功能模块设计

2.1 DHT11 传感器采集模块

DHT11 传感器采集模块负责温度与湿度数据的稳定触发、时序读取与校验解析。模块通过 GPIO 模式动态切换实现单总线双向通信，主机发送复位信号触发传感器响应，并在规定时序内读取 40bit 数据帧（湿度整数、湿度小数、温度整数、温度小数、校验和）。模块内置超时重试机制提高鲁棒性，并对接收到的数据进行 Checksum 校验，确保数据完整可靠。采集数据更新周期约 1 秒，可满足实时监测需求。该模块为系统数据源头，是后续串口上报与状态指示的基础。流程图如图 2.1 所示。

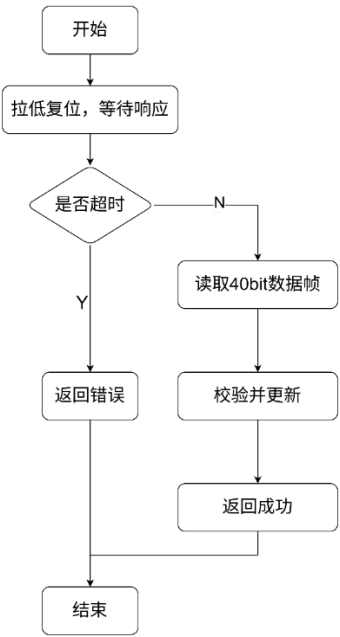


图 2.1 DHT11 传感器采集模块流程图

核心组件依赖主要有：

- 依赖外设：GPIOA Pin15、SysTick_Delay、单总线协议
- 依赖库：stm32f10x_gpio.h、Delay()、DHT11_Init()

关键实现如表 2.1 所示：

表 2.1 DHT11 传感器采集模块关键类与实现

关键函数	作用
DHT11_Rst()	发送复位信号
DHT11_Check()	检测传感器响应与超时

DHT11_Read_Bit()	读取单 bit 数据
DHT11_Read_Byte()	8bit 组帧
DHT11_Read_Data()	40bit 数据读取 + 校验解析

2.2 串口通信与数据上报模块（USART1）

串口通信模块基于 USART1 实现数据稳定发送与中断接收管理。模块以 115200 波特率运行，采用 8N1 无硬件流控模式。模块支持字节、数组、字符串、数字与格式化数据上报（printf 重定向），并维护 32Byte 循环缓冲区实现异步数据接收存储，防止数据丢失。USART_IT_RXNE 中断触发时自动写入缓冲区，主程序通过 GetByte 接口按需取出数据处理。该模块用于向上位机实时上报温湿度信息，是系统与外部交互的核心通道，确保监测数据可视化展示与进一步处理。流程图如图 2.2 所示。

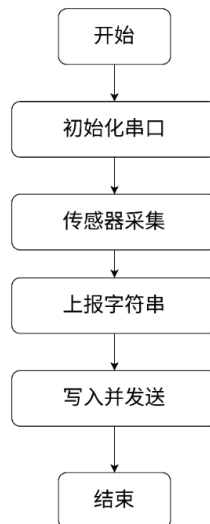


图 2.2 串口通信与数据上报模块（USART1）流程图

核心组件依赖主要有：

- 依赖外设：USART1 (PA9=TX, PA10=RX)
- 依赖库：stdio.h、stdarg.h、vsprintf()、NVIC 中断管理

关键实现如表 2.2 所示：

表 2.2 串口通信与数据上报模块关键类与实现

关键函数	作用
Serial_Init()	初始化 GPIO+USART+NVIC
Serial_SendByte()	阻塞发送字节
Serial_SendString()	发送字符串
Serial_Printf()	格式化上报

USART1_IRQHandler()	中断写入循环缓冲区
Serial_GetByte()	从缓冲区读取数据

2.3 LED 状态指示与系统心跳模块

LED 状态指示模块基于 GPIOB 控制板载 LED，实现系统运行状态反馈与视觉心跳指示。主程序每完成一次温湿度采集与串口上报后触发 LED 翻转，实现 1Hz 左右闪烁效果，直观表示系统处于正常运行状态。模块初始化时配置引脚为推挽输出模式，并提供 Toggle 接口供主循环调用。该模块虽结构简单，但在嵌入式监测系统中具有关键意义，可辅助快速判断系统是否卡死、采集是否正常进行、串口是否阻塞等运行状态问题，提高调试效率与系统可观测性。

核心组件依赖主要有：

- 依赖外设：GPIOB、AFIO(JTAG Disable)
- 依赖函数：LED_Init()、LED_Toggle()

关键实现如表 2.3 所示：

表 2.3 LED 状态指示与系统心跳模块关键类与实现

关键函数	作用
GPIO_ReadOutputDataBit	取反输出

2.4 SysTick 微秒延时与系统时基模块

主控制循环模块作为系统顶层任务调度中心，负责调用传感器采集、串口上报、LED 心跳翻转与延时节拍控制。模块采用阻塞式 1 秒周期调度策略（SysTick 提供微秒时基），循环调用 DHT11_Read_Data 获取数据，使用 Serial_Printf 格式化并通过 USART1 上报至上位机，同时执行 LED_Toggle 视觉心跳指示。该模块结构简洁、执行确定性强，适用于实时性要求高且任务逻辑明确的嵌入式监测系统，是系统稳定运行与数据交互的核心逻辑载体。

核心组件依赖主要有：

- 依赖外设：DHT11 采集、Serial 通信、LED 心跳、SysTick 时基
- 依赖中断：SysTick_Handler → TimingDelay_Decrement()

关键实现如表 2.4 所示：

表 2.4 SysTick 微秒延时与系统时基模块关键类与实现

关键函数	作用
------	----

SysTick_Config(HCLK/1e6)	1MHz 计数
TimingDelay_Decrement()	计数递减
Delay()	阻塞延时

2.5 主控制循环与任务调度模块

DHT11 传感器采集模块负责温度与湿度数据的稳定触发、时序读取与校验解析。模块通过 GPIO 模式动态切换实现单总线双向通信，主机发送复位信号触发传感器响应，并在规定时序内读取 40bit 数据帧（湿度整数、湿度小数、温度整数、温度小数、校验和）。模块内置超时重试机制提高鲁棒性，并对接收到的数据进行 Checksum 校验，确保数据完整可靠。采集数据更新周期约 1 秒，可满足实时监测需求。该模块为系统数据源头，是后续串口上报与状态指示的基础。流程图如图 2.3 所示。

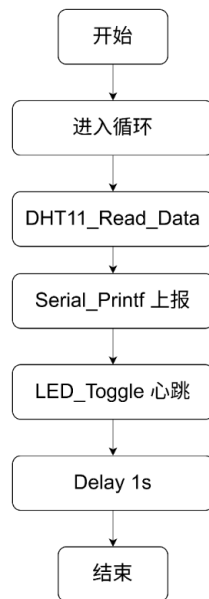


图 2.3 主控制循环与任务调度模块流程图

核心组件依赖主要有：

- 依赖模块：DHT11 采集、Serial 通信、LED 心跳、SysTick 时基
- 依赖函数：Delay()

关键实现为：

```

while(1){
    Read_Data → Printf → Toggle_LED → Delay(1s)
}
    
```

3 上位机功能模块设计

3.1 串口管理与数据接收模块

串口管理与数据接收模块基于 Qt SerialPort 实现串口设备扫描、连接控制与数据监听。模块初始化时自动加载可用串口端口并更新下拉列表，用户可选择端口与波特率进行通信。模块通过 readyRead 信号实现数据到达触发读取，支持实时解析传入的温度、湿度信息并剥离换行与回车干扰，保证数据稳定进入应用层。模块为系统数据输入核心通道，确保通信可开关、数据可清理、错误不阻塞主界面，是后续表格更新与图表绘制的前提基础。流程图如图 3.1 所示。

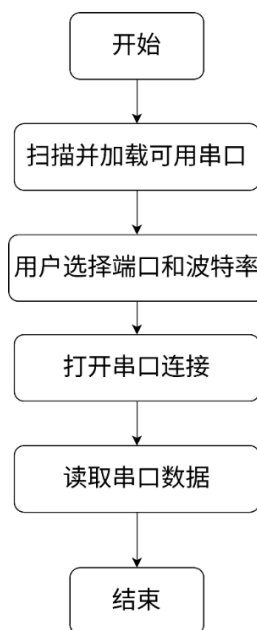


图 3.1 串口管理与数据接收模块流程图

核心组件依赖主要有：

- 依赖类：QSerialPort、QSerialPortInfo、QIODevice
- 依赖 UI 控件：QComboBox(cbSerialPort/cbBaudRate)、QPushButton(btnOpen/btnClose)

关键实现为：

```
class MainWidget {  
    QSerialPort *m_serial;  
    void initSerialPortList();
```



```
void on_btnOpen_clicked();
void on_btnClose_clicked();
void readSerialData();
}
```

3.2 数据展示与存储模块（表格更新）

数据展示模块负责将解析后的温湿度数据结构化显示到界面表格中，并维护历史数据存储容器。模块每收到一条有效数据即新增表格行，写入采集序号、温度、湿度及系统当前时间，实现数据自适应列宽显示便于查看。模块同时将数据追加到 QVector 数组用于图表重绘，并更新实时数据显示标签（当前温湿度、数据条数、历史极值），实现可视化监测与历史追踪能力，是系统人机交互中最直观的功能层。

核心组件依赖主要有：

- 依赖类：QTableWidget、QHeaderView、QVector
- 依赖 UI 控件：QLabel(labCurrentTemp/labCurrentHumi/labDataCount)、QTableWidget(tableData)

关键实现为：

```
tableData->insertRow()
tableData->setItem(row, col, new QTableWidgetItem())
m_temperature.append(temp)
m_humidity.append(humi)
```

3.3 历史曲线绘制模块（温湿度图表）

图表绘制模块负责对系统历史温度与湿度数据进行趋势可视化呈现。模块基于 QPainter 重写绘图事件，将温湿度统一映射至坐标系中，并分别绘制温度折线与湿度折线，同时标记温度为圆点、湿度为方点实现数据区分。模块支持数据更新后自动重绘，实现实时曲线增长与坐标刻度显示，具备图例标注温度与湿度含义。该模块为监测系统数据分析核心可视化层，直观展示环境变化趋势。流程图如图 3.2 所示。

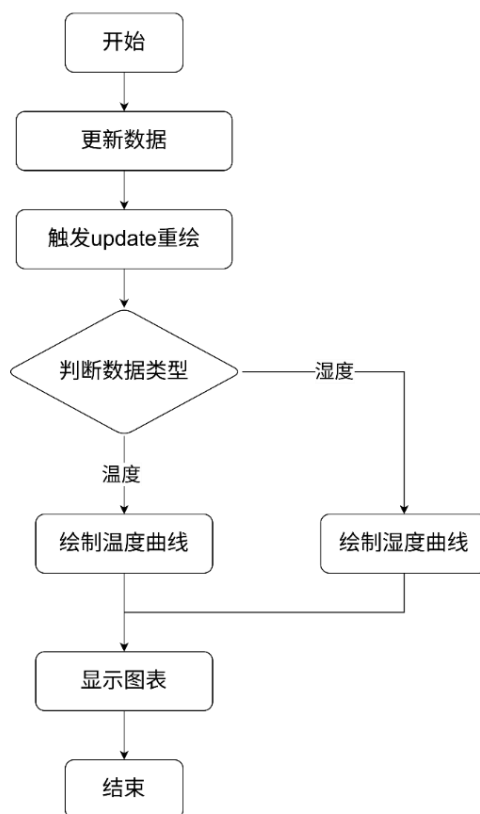


图 3.2 历史曲线绘制模块流程图

核心组件依赖主要有：

- 依赖类：QWidget、QPainter、QPen
- 依赖 UI 控件：QVector<int> index, QVector<float> temp, QVector<float> humi

关键实现为：

```

class ChartWidget : public QWidget {
    void setData(...)
    void paintEvent(...)
}
    
```

为实现对数据的长期保存，模块通过数据库接口将解析后的心电数据存储在 MySQL 数据库中。数据记录包括心电值、心率、血压以及时间戳，确保数据的完整性和可追溯性。数据库操作被设计为异步模式，避免影响可视化界面的实时性。

在模块开发中，特别注重异常处理和系统稳定性。无论是 TCP 连接中断、数据格式错误，还是数据库写入失败，系统均能有效捕获异常并记录日志，保证服务的连续性。

3.4 数据清理与系统重置模块

数据清理模块用于一键清空系统当前所有已接收与存储的温湿度记录，恢复系统初始监测状态。模块执行时删除表格全部行、清空温度与湿度历史数组、重置数据条数计数器，并恢复历史极值标记状态为第一条数据初始化模式，同时通知图表模块进行空数据重绘，避免旧曲线残留影响。该模块为系统提供快速重置能力，提高多次监测实验与串口调试效率。流程图如图 3.3 所示。

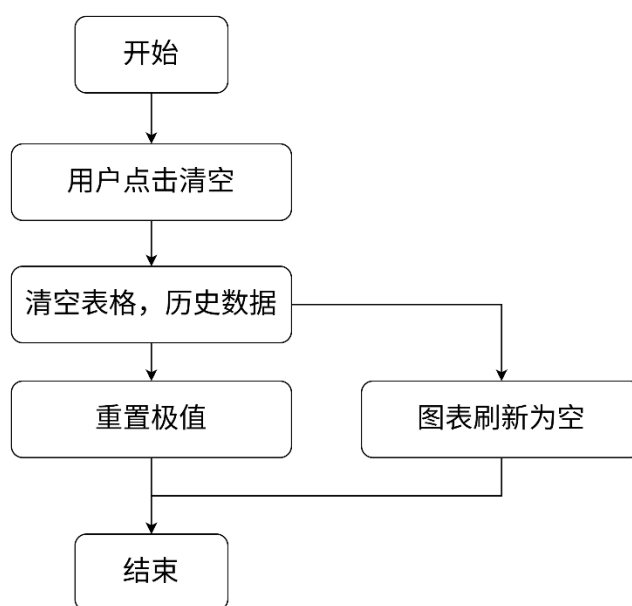


图 3.3 数据清理模块流程图

核心组件依赖主要有：

- 依赖类：QVector.clear()、QTableWidget.setRowCount()、ChartWidget.setData()
- 依赖 UI 控件：QPushButton(btnClear)、QTableWidget(tableData)

关键实现为：

```
tableData->setRowCount(0)
```

```
m_temperature.clear()
```

```
m_humidity.clear()
```

```
chartWidget->setData(...)
```

```
reset extreme flags
```

3.5 主界面控制与模块联调中心

主界面控制模块负责整合串口、表格、图表与清理模块，实现系统级功能联调。模块初始化加载串口列表并创建图表容器，用户通过按钮控制串口开关与数据清理。系统运行中数据读取驱动表格新增、曲线增长与极值刷新。模块逻辑链路清晰，运行节拍由串口信号驱动，是系统功能稳定运行的调度中心。流程图如图 3.4 所示。

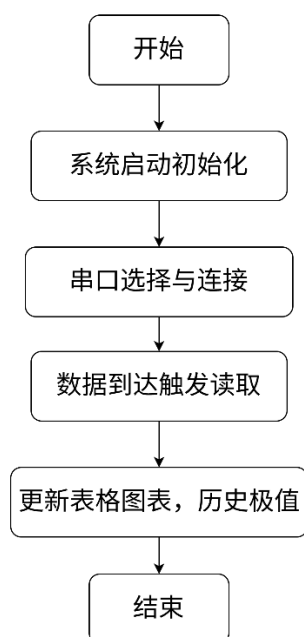


图 3.4 主界面控制与模块联调中心流程图

核心组件依赖主要有：

- 依赖模块：串口接收、表格展示、图表绘制、数据清理
- 依赖类：QWidget、QApplication

关键实现为：

`connect(serial.readyRead → readSerialData)`

`readData → table update → chart update → extreme update`

4 实现方案

// dht11.c 文件:

```
#include "dht11.h"
```

```
#include "main.h"
```

```
void DHT11_IO_OUT(void) {
```

```
    GPIO_InitTypeDef  GPIO_InitStructure;
```

```
    GPIO_InitStructure.GPIO_Pin = DHT11_GPIO_PIN;
```

```
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
```

```
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
```

```
    GPIO_Init(DHT11_GPIO_PORT, &GPIO_InitStructure);
```

```
}
```

```
void DHT11_IO_IN(void) {
```

```
    GPIO_InitTypeDef  GPIO_InitStructure;
```

```
    GPIO_InitStructure.GPIO_Pin = DHT11_GPIO_PIN;
```

```
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;
```

```
    GPIO_Init(DHT11_GPIO_PORT, &GPIO_InitStructure);
```

```
}
```

```
//, 'Î»DHT11
```

```
void DHT11_Rst(void) {
```

```
    DHT11_IO_OUT();    //SET OUTPUT
```

```
    GPIO_ResetBits(DHT11_GPIO_PORT, DHT11_GPIO_PIN);
```

```
    Delay(20000);       //À-µÍÖÁËÙ18ms
```

```
    GPIO_SetBits(DHT11_GPIO_PORT, DHT11_GPIO_PIN);
```

```
    Delay(30);          //Ö÷»úÀ-, ß20~40us
```

```

}

//µÈ' ýDHT11µÄ»00!
// • µ»01:Î'¼1² âµ½DHT11µÄ' æÔÚ
// • µ»00:' æÔÚ

u8 DHT11_Check(void) {
    u8 retry = 0;
    DHT11_IO_IN(); //SET INPUT
    while (GPIO_ReadInputDataBit(DHT11_GPIO_PORT, DHT11_GPIO_PIN)
&& retry < 100) { //DHT11»áÀ-µÍ40~80us
        retry++;
        Delay(1);
    };
    if (retry >= 100)
        return 1;
    else
        retry = 0;
    while (!GPIO_ReadInputDataBit(DHT11_GPIO_PORT, DHT11_GPIO_PIN)
&& retry < 100) { //DHT11À-µÍ° ó»áÔÛ' ÎÀ-, ß40~80us
        retry++;
        Delay(1);
    };
    if (retry >= 100)
        return 1;
    return 0;
}

//' ÓDHT11¶ÁÈ;Ò», öÎ»
// • µ»00µ£° 1/0

u8 DHT11_Read_Bit(void) {
    u8 retry = 0;

```

```

        while (GPIO_ReadInputDataBit(DHT11_GPIO_PORT, DHT11_GPIO_PIN)
&& retry < 100) { //µÈ'ý±äîªµÍµçÆ½
            retry++;
            Delay(1);
        }
        retry = 0;
        while (!GPIO_ReadInputDataBit(DHT11_GPIO_PORT, DHT11_GPIO_PIN)
&& retry < 100) { //µÈ'ý±ä,ßµçÆ½
            retry++;
            Delay(1);
        }
        Delay(40); //µÈ'ý40us
        if (GPIO_ReadInputDataBit(DHT11_GPIO_PORT, DHT11_GPIO_PIN))
            return 1;
        else
            return 0;
    }
    //´ÓDHT11¶ÁÈìÒ»¸ö×ÖÖÚ
    //•µ»ØÖµ£°¶Áµ½µÄËý³Ý
    u8 DHT11_Read_Byte(void) {
        u8 i, dat;
        dat = 0;
        for (i = 0; i < 8; i++) {
            dat <<= 1;
            dat |= DHT11_Read_Bit();
        }
        return dat;
    }
    //´ÓDHT11¶ÁÈìÒ»ÊËý³Ý

```

```

//temp:Â°ËÖµ( • °Ë § :0~50;ã)
//humi:Ê°ËÖµ( • °Ë § :20%~90%)
// • µ»00µ£° 0, Õý³ £;1, °ÁÈ;Ê § ° Ü
u8 DHT11_Read_Data(u8 *temp, u8 *humi) {
    u8 buf[5];
    u8 i;
    DHT11_Rst();
    if (DHT11_Check() == 0) {
        for (i = 0; i < 5; i++) { //°ÁÈ;40Î»Êý¼Ý
            buf[i] = DHT11_Read_Byte();
        }
        if ((buf[0] + buf[1] + buf[2] + buf[3]) == buf[4]) {
            *humi = buf[0];
            *temp = buf[2];
        }
    } else
        return 1;
    return 0;
}

//³ òÊ¼» DHT11µÄIO¿Ú DQ Í¬Ê±¼²âDHT11µÄæÔÚ
// • µ»01:²»æÔÚ
// • µ»00:æÔÚ
u8 DHT11_Init(void) {

    DHT11_Rst(); //, 'Î»DHT11
    return DHT11_Check(); //µÈýDHT11µÄ»00!
}
    
```

```

// dht11.h 文件:

#ifndef __DHT11_H
#define __DHT11_H

#include "stm32f10x.h"

#define DHT11_GPIO_PORT    GPIOA

//GPIO引脚定义
#define DHT11_GPIO_CLK      RCC_APB2Periph_GPIOA

//GPIO引脚名称
#define DHT11_GPIO_PIN      GPIO_Pin_15

//A-40μsSCL±0.5μsGPIO

u8 DHT11_Init(void); //DHT11
u8 DHT11_Read_Data(u8 *temp, u8 *humi); //读取温度湿度
u8 DHT11_Read_Byte(void); //读取一个字节
u8 DHT11_Read_Bit(void); //读取一位
u8 DHT11_Check(void); //校验DHT11
void DHT11_Rst(void); //复位DHT11

#endif

// serial.c 文件:

#include "Serial.h"

/**
 * 函 数: 串口初始化
 * 参 数: 无
 * 返 回 值: 无
 */
void Serial_Init(void) {

```

```

/*开启时钟*/
RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1, ENABLE); //开启
USART1 的时钟
RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE); // 开 启
GPIOA 的时钟

/*GPIO 初始化*/
GPIO_InitTypeDef GPIO_InitStructure;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
GPIO_Init(GPIOA, &GPIO_InitStructure); //将 PA9 引脚初
始化为复用推挽输出

GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
GPIO_Init(GPIOA, &GPIO_InitStructure); //将 PA10 引脚初
始化为上拉输入

/*USART 初始化*/
USART_InitTypeDef USART_InitStructure; //定义结构体变
量
USART_InitStructure.USART_BaudRate = 115200; //波特率
USART_InitStructure.USART_HardwareFlowControl =
USART_HardwareFlowControl_None; //硬件流控制, 不需要
USART_InitStructure.USART_Mode = USART_Mode_Tx | USART_Mode_Rx;
//模式, 发送模式和接收模式均选择
USART_InitStructure.USART_Parity = USART_Parity_No; //奇偶校

```

验，不需要

```
    USART_InitStructure.USART_StopBits = USART_StopBits_1;    //停止
    位，选择 1 位
```

```
    USART_InitStructure.USART_WordLength = USART_WordLength_8b;
    //字长，选择 8 位
```

```
    USART_Init(USART1, &USART_InitStructure);                //将结构体变量
    交给 USART_Init，配置 USART1
```

```
    /*中断输出配置*/
```

```
    USART_ITConfig(USART1, USART_IT_RXNE, ENABLE);            //开启串口
    接收数据的中断
```

```
    /*NVIC 中断分组*/
```

```
    //NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);          //配置
    NVIC 为分组 2
```

```
    /*NVIC 配置*/
```

```
    NVIC_InitTypeDef NVIC_InitStructure;                        //定义结构体变量
    NVIC_InitStructure.NVIC_IRQChannel = USART1_IRQn;          //选择配置
    NVIC 的 USART1 线
```

```
    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;            //指定 NVIC
    线路使能
```

```
    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 1;    //
    指定 NVIC 线路的抢占优先级为 1
```

```
    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;          //指定
    NVIC 线路的响应优先级为 1
```

```
    NVIC_Init(&NVIC_InitStructure);                            //将结构体变量交给
    NVIC_Init，配置 NVIC 外设
```

```

    /*USART 使能*/

    USART_Cmd(USART1, ENABLE);           //使能 USART1，串口开
始运行
}

/**
 * 函    数：串口发送一个字节
 * 参    数：Byte 要发送的一个字节
 * 返 回 值：无
 */
void Serial_SendByte(uint8_t Byte) {
    USART_SendData(USART1, Byte);    //将字节数据写入数据寄存器，写
入后 USART 自动生成时序波形
    while (USART_GetFlagStatus(USART1, USART_FLAG_TXE) == RESET); //
等待发送完成
    /*下次写入数据寄存器会自动清除发送完成标志位，故此循环后，无需清
除标志位*/
}

/**
 * 函    数：串口发送一个数组
 * 参    数：Array 要发送数组的首地址
 * 参    数：Length 要发送数组的长度
 * 返 回 值：无
 */
void Serial_SendArray(uint8_t* Array, uint16_t Length) {
    uint16_t i;
    for (i = 0; i < Length; i ++) { //遍历数组
        Serial_SendByte(Array[i]);    //依次调用 Serial_SendByte 发送

```

每个字节数据

```
    }
}
```

```
/**
```

```
 * 函    数：串口发送一个字符串
```

```
 * 参    数：String 要发送字符串的首地址
```

```
 * 返 回 值：无
```

```
 */
```

```
void Serial_SendString(char* String) {
```

```
    uint8_t i;
```

```
    for (i = 0; String[i] != '\0'; i++) { //遍历字符数组（字符串），
遇到字符串结束标志位后停止
```

```
        Serial_SendByte(String[i]);    //依次调用 Serial_SendByte 发送
```

每个字节数据

```
    }
}
```

```
/**
```

```
 * 函    数：次方函数（内部使用）
```

```
 * 返 回 值：返回值等于 X 的 Y 次方
```

```
 */
```

```
uint32_t Serial_Pow(uint32_t X, uint32_t Y) {
```

```
    uint32_t Result = 1; //设置结果初值为 1
```

```
    while (Y --) {    //执行 Y 次
```

```
        Result *= X;    //将 X 累乘到结果
```

```
    }
```

```
    return Result;
```

```
}
```

```

/**
 * 函 数：串口发送数字
 * 参 数：Number 要发送的数字，范围：0~4294967295
 * 参 数：Length 要发送数字的长度，范围：0~10
 * 返 回 值：无
 */
void Serial_SendNumber(uint32_t Number, uint8_t Length) {
    uint8_t i;
    for (i = 0; i < Length; i ++) { //根据数字长度遍历数字的每一位
        Serial_SendByte(Number / Serial_Pow(10, Length - i - 1) % 10
+ '0'); //依次调用 Serial_SendByte 发送每位数字
    }
}

/**
 * 函 数：使用 printf 需要重定向的底层函数
 * 参 数：保持原始格式即可，无需变动
 * 返 回 值：保持原始格式即可，无需变动
 */
int fputc(int ch, FILE *f) {
    Serial_SendByte(ch); //将 printf 的底层重定向到自己的发送字
节函数
    return ch;
}

/**
 * 函 数：自己封装的 printf 函数
 * 参 数：format 格式化字符串

```

```

    * 参    数: ... 可变的参数列表
    * 返 回 值: 无
    */

void Serial_Printf(char* format, ...) {
    char String[100];          //定义字符数组
    va_list arg;               //定义可变参数列表数据类型的变量 arg
    va_start(arg, format);     //从 format 开始, 接收参数列表到 arg
变量
    vsprintf(String, format, arg); //使用 vsprintf 打印格式化字符串
和参数列表到字符数组中
    va_end(arg);               //结束变量 arg
    Serial_SendString(String);  //串口发送字符数组 (字符串)
}

static char Serial_RxBuffer[Serial_RxBufLength];
static uint16_t rxStartPos = 0, rxEndPos = 0, rxSize = 0;

int16_t Serial_GetByte(void) {
    int16_t bt = -1;
    if (rxSize > 0) {
        bt = Serial_RxBuffer[rxStartPos++];
        rxStartPos %= Serial_RxBufLength;
        rxSize--;
    }
    return bt;
}

/**
    * 函    数: USART1 中断函数
    * 参    数: 无

```

* 返回值：无

* 注意事项：此函数为中断函数，无需调用，中断触发后自动执行

* 函数名为预留的指定名称，可以从启动文件复制

* 请确保函数名正确，不能有任何差异，否则中断函数将不能进入

```

*/
void USART1_IRQHandler(void) {

    if (USART_GetITStatus(USART1, USART_IT_RXNE) == SET) {
        if (rxSize < Serial_RxBufLength) {
            Serial_RxBuffer[rxEndPos++] = USART_ReceiveData(USART1);
            rxEndPos %= Serial_RxBufLength;
            rxSize++;
        }
        USART_ClearITPendingBit(USART1, USART_IT_RXNE);    //清除标志
    }
}

```

```

// serial.h 文件：
#ifndef __SERIAL_H
#define __SERIAL_H

#include "stm32f10x.h"
#include <stdio.h>
#include <stdarg.h>

#define Serial_RxBufLength 32

```



```

void Serial_Init(void);
void Serial_SendByte(uint8_t Byte);
void Serial_SendArray(uint8_t* Array, uint16_t Length);
void Serial_SendString(char* String);
void Serial_SendNumber(uint32_t Number, uint8_t Length);
void Serial_Printf(char* format, ...);
int16_t Serial_GetByte(void);
#endif

// main.c 文件:
#include "stm32f10x.h"
#include <stdio.h>
#include "main.h"
#include "../led/led.h"
#include "../serial/serial.h"
#include "../dht11/dht11.h"
static __IO uint32_t TimingDelay;
RCC_ClocksTypeDef RCC_Clocks;
u8 temperature;
u8 humidity;
uint32_t n = 0;
int main(void) {
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB
RCC_APB2Periph_AFIO, ENABLE);
    GPIO_PinRemapConfig(GPIO_Remap_SWJ_JTAGDisable, ENABLE);

    RCC_GetClocksFreq(&RCC_Clocks);
    SysTick_Config(RCC_Clocks.HCLK_Frequency / 1000000);

```

```

    LED_Init();
    DHT11_Init();
    Serial_Init();

    while (1) {
        DHT11_Read_Data(&temperature, &humidity); //¶ÁÈ; îÊª ¶ÈÖµ
        Serial_Printf("No. %u   îÊª ¶È: %d; æ      Êª ¶È: %d%%RH\r\n",    n++,
temperature, humidity);
        LED_Toggle();
        Delay(1000000);

    }
}

void Delay(__IO uint32_t nTime) {
    TimingDelay = nTime;
    while (TimingDelay != 0);
}

void TimingDelay_Decrement(void) {
    if (TimingDelay != 0x00) {
        TimingDelay--;
    }
}

// main.h 文件:
#ifndef __MAIN_H
#define __MAIN_H

void Delay(__IO uint32_t nTime);
void TimingDelay_Decrement(void);

```

```
#endif

// DHT11SerialTool.pro:
QT += core gui widgets serialport
CONFIG += c++11

SOURCES += \
    main.cpp \
    mainwidget.cpp \
    ChartWidget.cpp

HEADERS += \
    mainwidget.h \
    ChartWidget.h

FORMS += \
    mainwidget.ui

# 部署配置
qnx: target.path = /tmp/${TARGET}/bin
else: unix:!android: target.path = /opt/${TARGET}/bin
!isEmpty(target.path): INSTALLS += target

// ChartWidget.h 文件:
#ifndef CHARTWIDGET_H
#define CHARTWIDGET_H

#include <QWidget>
```

```
#include <QVector>

class ChartWidget : public QWidget
{
    Q_OBJECT
public:
    explicit ChartWidget(QWidget *parent = nullptr);
    // 湿度参数
    void setData(const QVector<int>& index, const QVector<float>&
temp, const QVector<float>& humi);

protected:
    void paintEvent(QPaintEvent *event) override;

private:
    QVector<int> m_index;
    QVector<float> m_temp;
    QVector<float> m_humi; // 湿度数据
};

#endif // CHARTWIDGET_H

// mainwidget.h 文件:
#ifndef MAINWIDGET_H
#define MAINWIDGET_H

#include <QWidget>
#include <QSerialPort>
#include <QVector>
```

```
// 前置声明

class ChartWidget;

namespace Ui {
class MainWidget;
}

class MainWidget : public QWidget
{
    Q_OBJECT

public:
    explicit MainWidget(QWidget *parent = nullptr);
    ~MainWidget();

private slots:
    void on_btnOpen_clicked();
    void on_btnClose_clicked();
    void on_btnClear_clicked();
    void readSerialData();

private:
    Ui::MainWidget *ui;
    QSerialPort *m_serial;
    int m_dataCount;
    QVector<int> m_index;           // 序号数据
    QVector<float> m_temperature; // 温度数据
    QVector<float> m_humidity;    // 湿度数据
    float m_maxTemp; // 历史最高温度
    float m_minTemp; // 历史最低温度
}
```

```

float m_maxHumi; // 历史最高湿度
float m_minHumi; // 历史最低湿度
bool m_isFirstData; // 标记是否是第一条数据
ChartWidget *m_chartWidget;

void initSerialPortList();
void initChartWidget();
void updateExtremeValues(float newTemp, float newHumi); // 更新历史极值
};

#endif // MAINWIDGET_H

// ChartWidget.cpp 文件:
#include "ChartWidget.h"
#include <QPainter>
#include <QFont>

ChartWidget::ChartWidget(QWidget *parent) : QWidget(parent)
{
    this->setStyleSheet("background-color: white;");
}

// 接收温度+湿度数据
void ChartWidget::setData(const QVector<int>& index, const
QVector<float>& temp, const QVector<float>& humi)
{
    m_index = index;
    m_temp = temp;

```

```

        m_humi = humi; // 保存湿度数据
        this->update(); // 触发重绘
    }

// 重写绘图事件
void ChartWidget::paintEvent(QPaintEvent *event)
{
    Q_UNUSED(event);
    if (m_index.isEmpty()) return;

    QPainter painter(this);
    painter.setRenderHint(QPainter::Antialiasing);

    // 绘图区域
    QRect rect = this->rect().adjusted(50, 30, -30, -30);
    int w = rect.width();
    int h = rect.height();

    // 坐标轴范围
    int xMin = 0;
    int xMax = m_index.last() + 2;
    float yMin = 0;
    float yMax = 100;

    // 绘制坐标轴
    painter.setPen(QPen(Qt::black, 1));
    painter.drawLine(rect.bottomLeft(), rect.topLeft()); // Y 轴
    painter.drawLine(rect.bottomLeft(), rect.bottomRight()); // X

```

轴

```

// 绘制 X 轴刻度
for (int x = xMin; x <= xMax; x += 2) {
    float xPos = rect.left() + (x - xMin) * w / (xMax - xMin);
    painter.drawLine(xPos, rect.bottom(), xPos, rect.bottom()
+ 5);

    painter.drawText(xPos - 10, rect.bottom() + 20,
QString::number(x));
}

// 绘制 Y 轴刻度
for (int y = yMin; y <= yMax; y += 10) {
    float yPos = rect.bottom() - (y - yMin) * h / (yMax - yMin);
    painter.drawLine(rect.left() - 5, yPos, rect.left(), yPos);
    painter.drawText(rect.left() - 30, yPos + 5,
QString::number(y));
}

// 绘制温度曲线（红色）
painter.setPen(QPen(Qt::red, 2));
for (int i = 1; i < m_index.size(); ++i) {
    float x1 = rect.left() + (m_index[i-1] - xMin) * w / (xMax
- xMin);

    float y1 = rect.bottom() - (m_temp[i-1] - yMin) * h / (yMax
- yMin);

    float x2 = rect.left() + (m_index[i] - xMin) * w / (xMax
- xMin);

    float y2 = rect.bottom() - (m_temp[i] - yMin) * h / (yMax
- yMin);

```

```

        painter.drawLine(x1, y1, x2, y2);
        painter.drawEllipse(QPointF(x2, y2), 3, 3); // 温度数据点
        (圆形)
    }

    // 绘制湿度曲线 (蓝色)
    painter.setPen(QPen(Qt::blue, 2));
    for (int i = 1; i < m_index.size(); ++i) {
        float x1 = rect.left() + (m_index[i-1] - xMin) * w / (xMax
- xMin);
        float y1 = rect.bottom() - (m_humi[i-1] - yMin) * h / (yMax
- yMin);
        float x2 = rect.left() + (m_index[i] - xMin) * w / (xMax
- xMin);
        float y2 = rect.bottom() - (m_humi[i] - yMin) * h / (yMax
- yMin);
        painter.drawLine(x1, y1, x2, y2);
        painter.drawRect(QRectF(x2-3, y2-3, 6, 6)); // 湿度数据点
        (方形)
    }

    // 图例 (区分温度/湿度)
    painter.setPen(Qt::red);
    painter.drawText(rect.right() - 150, rect.top() + 20, "温度
(℃)");
    painter.setPen(Qt::blue);
    painter.drawText(rect.right() - 50, rect.top() + 20, "湿度
(%)");
}

```

```
// main.cpp 文件:
```

```
#include "mainwindow.h"
```

```
#include <QApplication>
```

```
int main(int argc, char *argv[])
```

```
{
```

```
    QApplication a(argc, argv);
```

```
    MainWindow w;
```

```
    w.show();
```

```
    return a.exec();
```

```
}
```

```
// mainwidget.cpp 文件:
```

```
#include "mainwindow.h"
```

```
#include "ui_mainwidget.h"
```

```
#include "ChartWidget.h"
```

```
#include <QSerialPortInfo>
```

```
#include <QMessageBox>
```

```
#include <QDateTime>
```

```
#include <QVBoxLayout>
```

```
#include <QDateTime>
```

```
#include <QMessageBox>
```

```
MainWindow::MainWindow(QWidget *parent) :
```

```
    QWidget(parent),
```

```
    ui(new Ui::MainWindow),
```

```
    m_serial(new QSerialPort(this)),
```

```
    m_dataCount(0),
```

```
    m_maxTemp(0.0),
```

```

        m_minTemp(0.0),
        m_maxHumi(0.0),
        m_minHumi(0.0),
        m_isFirstData(true),
        m_chartWidget(nullptr)
    {
        ui->setupUi(this);
        //表格自适应

ui->tableData->horizontalHeader()->setSectionResizeMode(QHeaderView::
ResizeToContents);

        // 初始化显示控件默认值
        ui->labDataCount->setText("0 条");
        ui->labCurrentTemp->setText("暂无");
        ui->labCurrentHumi->setText("暂无");
        ui->labMaxTemp->setText("暂无");
        ui->labMinTemp->setText("暂无");
        ui->labMaxHumi->setText("暂无");
        ui->labMinHumi->setText("暂无");
        // 初始化串口列表和绘图控件
        initSerialPortList();
        initChartWidget();

        // 信号槽连接
        connect(m_serial,          &QSerialPort::readyRead,          this,
&MainWindow::readSerialData);
    }

```

```
MainWidget::~MainWidget()
{
    if (m_serial->isOpen()) m_serial->close();
    delete ui;
}

// 初始化串口列表
void MainWidget::initSerialPortList()
{
    ui->cbSerialPort->clear();
    for (const auto& port : QSerialPortInfo::availablePorts()) {
        if (!port.manufacturer().isEmpty()) {
            ui->cbSerialPort->addItem(port.portName());
        }
    }
    if (ui->cbSerialPort->count() == 0) {
        ui->cbSerialPort->addItem("无可使用串口");
        ui->btnOpen->setEnabled(false);
    }
}

// 初始化绘图控件（嵌入到 widgetChart 容器）
void MainWidget::initChartWidget()
{
    m_chartWidget = new ChartWidget();
    QVBoxLayout *layout = new QVBoxLayout(ui->widgetChart);
    layout->setContentsMargins(0, 0, 0, 0);
    layout->addWidget(m_chartWidget);
}
```

```
// 打开串口

void MainWindow::on_btnOpen_clicked()
{
    m_serial->setPortName(ui->cbSerialPort->currentText());
    m_serial->setBaudRate(ui->cbBaudRate->currentText().toInt());
    m_serial->setDataBits(QSerialPort::Data8);
    m_serial->setParity(QSerialPort::NoParity);
    m_serial->setStopBits(QSerialPort::OneStop);

    if (m_serial->open(QIODevice::ReadWrite)) {
        ui->btnOpen->setEnabled(false);
        ui->btnClose->setEnabled(true);
        QMessageBox::information(this, "成功", "串口已打开");
    } else {
        QMessageBox::critical(this, "失败", "串口打开失败");
    }
}

// 关闭串口

void MainWindow::on_btnClose_clicked()
{
    m_serial->close();
    ui->btnOpen->setEnabled(true);
    ui->btnClose->setEnabled(false);
}

// 更新历史最高/最低温湿度

void MainWindow::updateExtremeValues(float newTemp, float newHumi)
{

```

```

// 第一条数据：直接初始化极值
if (m_isFirstData) {
    m_maxTemp = newTemp;
    m_minTemp = newTemp;
    m_maxHumi = newHumi;
    m_minHumi = newHumi;
    m_isFirstData = false; // 标记为非第一条数据
} else {
    // 非第一条数据：对比更新极值

    if (newTemp > m_maxTemp) m_maxTemp = newTemp; // 更新最高
温度

    if (newTemp < m_minTemp) m_minTemp = newTemp; // 更新最低
温度

    if (newHumi > m_maxHumi) m_maxHumi = newHumi; // 更新最高
湿度

    if (newHumi < m_minHumi) m_minHumi = newHumi; // 更新最低
湿度

}

// ===== 刷新界面显示极值 =====
ui->labMaxTemp->setText(QString("%1℃").arg(m_maxTemp, 0, 'f',
1));
ui->labMinTemp->setText(QString("%1℃").arg(m_minTemp, 0, 'f',
1));
ui->labMaxHumi->setText(QString("%1%").arg(m_maxHumi, 0, 'f',
1));
ui->labMinHumi->setText(QString("%1%").arg(m_minHumi, 0, 'f',
1));
}

```

```
// 清空数据

void MainWindow::on_btnClear_clicked()
{
    ui->tableData->setRowCount(0);
    m_dataCount = 0;
    m_index.clear();
    m_temperature.clear();
    m_humidity.clear();
    m_chartWidget->setData(m_index, m_temperature, m_humidity);
    // 清空时重置显示控件
    ui->labDataCount->setText("0 条");
    ui->labCurrentTemp->setText("暂无");
    ui->labCurrentHumi->setText("暂无");
    // 重置极值变量和显示
    m_maxTemp = 0.0;
    m_minTemp = 0.0;
    m_maxHumi = 0.0;
    m_minHumi = 0.0;
    m_isFirstData = true; // 恢复为第一条数据标记
    ui->labMaxTemp->setText("暂无");
    ui->labMinTemp->setText("暂无");
    ui->labMaxHumi->setText("暂无");
    ui->labMinHumi->setText("暂无");
}
```

```
// 读取串口数据

void MainWindow::readSerialData()
```

```

{
    // 1. 读取并清理数据

    QByteArray data = m_serial->readAll();

    QString dataStr = QString::fromUtf8(data).replace("\r",
    "").replace("\n", "").trimmed();

    if (dataStr.isEmpty()) return;

    // 2. 解析温湿度

    QStringList dataList = dataStr.split(",");

    /*if (dataList.size() != 2) {
        QMessageBox::warning(this, "错误", "数据格式错误！正确格式：
    温度,湿度（如 25.5,60）");
        return;
    }*/

    bool tempOk, humiOk;
    float temp = dataList[0].toFloat(&tempOk);
    float humi = dataList[1].toFloat(&humiOk);

    /*if (!tempOk || !humiOk) {
        QMessageBox::warning(this, "错误", "温度/湿度不是有效数字！
    ");
        return;
    }*/

    //更新表格
    m_dataCount++;

    int row = ui->tableData->rowCount();
    ui->tableData->insertRow(row);

    // 序号列
    ui->tableData->setItem(row, 0, new
    
```

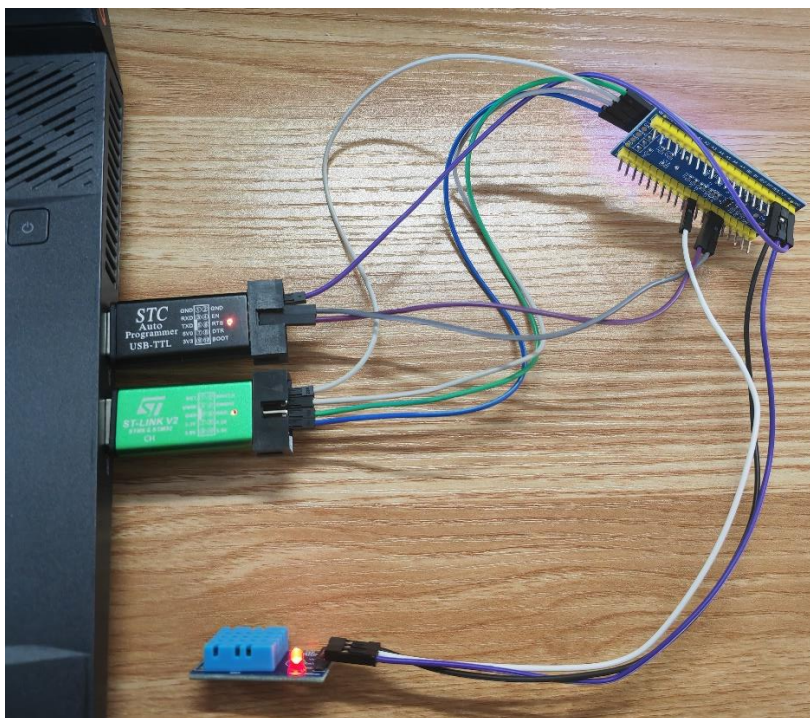
```

QTableWidgetItem(QString::number(m_dataCount)));
    // 温度列
    ui->tableData->setItem(row, 1, new
QTableWidgetItem(QString("%1℃").arg(temp)));
    // 湿度列
    ui->tableData->setItem(row, 2, new
QTableWidgetItem(QString("%1%").arg(humi)));
    // 采集时间列
    QString currentTime =
QDateTime::currentDateTime().toString("yyyy-MM-dd hh:mm:ss");
    ui->tableData->setItem(row, 3, new
QTableWidgetItem(currentTime));
    // 更新实时显示控件
    ui->labDataCount->setText(QString("%1 条").arg(m_dataCount));
// 数据条数
    ui->labCurrentTemp->setText(QString("%1℃").arg(temp, 0, 'f',
1)); // 温度
    ui->labCurrentHumi->setText(QString("%1%").arg(humi, 0, 'f',
1)); // 湿度
    updateExtremeValues(temp, humi);
    //更新绘图数据
    m_index.append(m_dataCount);
    m_temperature.append(temp);
    m_humidity.append(humi);
    m_chartWidget->setData(m_index, m_temperature, m_humidity);
}
    
```

5.1 硬件调试

GPIO 通信调试：检查 STM32 的 PA15 引脚与 DHT11 的 DQ 线连接是否牢固可靠，焊点无松动或接触不良。必要时轻拉排线、按压连接器确认无断连风险，避免因引脚松动导致读取失败或信号波动，确保数据线在运行中可保持稳定的高低电平切换能力。

串口硬件连接检查：核对 PA9（TX）与上位机 RX、PA10（RX）与上位机 TX 是否完成交叉连接，确认串口线序、杜邦线或 USB 转 TTL 模块接口无接反。调试目标是确保 STM32 串口物理链路正确、通信方向无误，避免因直连或线序错误导致上位机无法稳定接收数据。



Keil 程序运行验证：Keil 程序在线烧录运行固件后，通过调试接口进入实时

运行验证阶段。重点观察开发板 LED 是否保持稳定的周期性翻转，作为系统心跳指示，辅助判断主循环是否正常调度。该步骤用于综合验证硬件电路与固件逻辑联调成功，确保系统已稳定进入主循环数据采集与上报状态。程序烧录结果如图 5.2 所示。

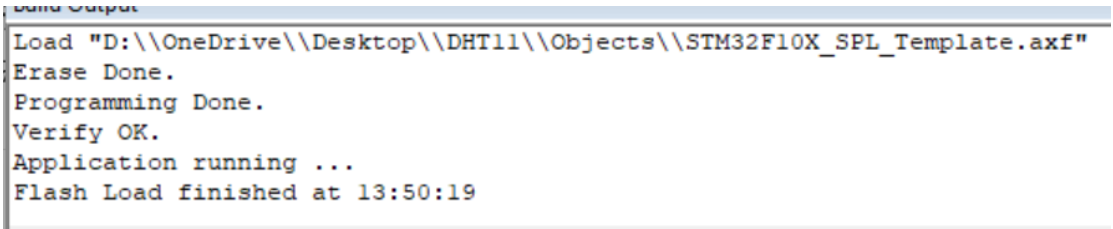


图 5.2 程序烧录结果

数据传输质量检测：通过串口助手连续接收多条数据，观察传输效果与刷新节奏，验证数据上报是否符合预期通信周期。验证结果如图 5.3 所示。

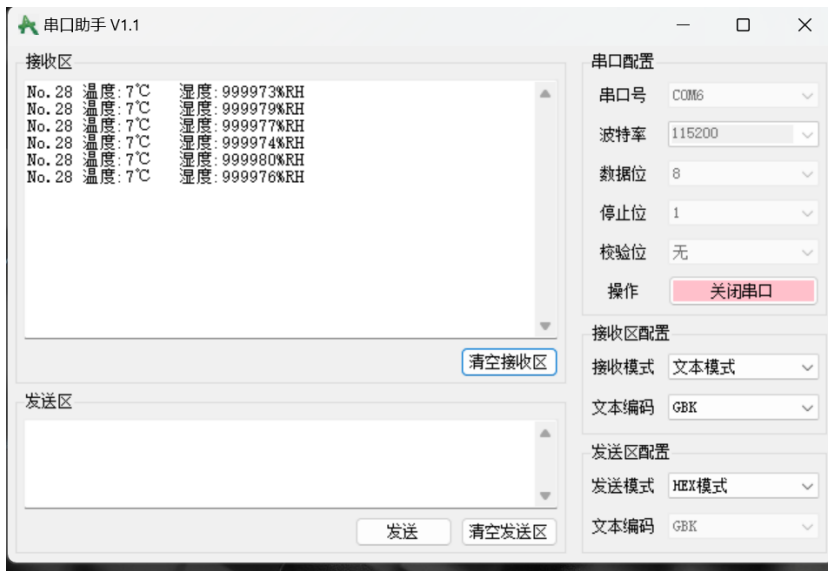


图 5.3 数据传输验证结果

5.2 软件调试

Qt 串口功能调试：运行上位机软件，验证是否能正确枚举串口端口、完成打开与关闭操作，并稳定触发 `readyRead` 读取回调。调试目标是确保串口模块在 Qt 环境中初始化与运行正常，接口响应及时稳定，为后续数据解析与显示提供可靠的数据接收入口。串口打开操作效果如图 5.4 所示，串口关闭操作效果如图 5.5 所示。



图 5.5 串口打开操作效果

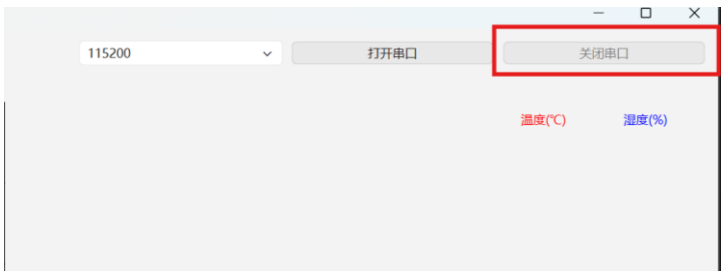


图 5.5 串口关闭操作效果

数据接收完整性调试：通过下位机发送标准格式字符串，验证上位机是否能完整接收每条数据，无堆叠、截断或遗漏。此步骤用于确认上位机的接收缓冲与读取逻辑稳定可靠，避免数据丢失或解析失败，确保通信链路在软件层面无阻塞风险。数据接收操作效果如图 5.6 所示。

The screenshot shows the '串口' (Serial Port) dropdown menu set to 'COM6', which is highlighted with a red rectangle. Below this, a table displays the received data. The table has four columns: '序号' (Serial Number), '温度' (Temperature), '湿度' (Humidity), and '时间' (Time). The data is as follows:

序号	温度	湿度	时间
1	25°C	7%	2025-12-29 15:24:33
2	25°C	7%	2025-12-29 15:24:36
3	25°C	7%	2025-12-29 15:24:39
4	25°C	7%	2025-12-29 15:24:42
5	25°C	7%	2025-12-29 15:24:45
6	25°C	7%	2025-12-29 15:24:48

图 5.6 数据接收操作效果

数据解析功能调试：检查接收数据是否可被正确分割并提取温湿度数值，更新至 UI 标签与数据表格。调试重点在于验证解析逻辑容错稳定、数值转换无异常、格式识别准确，避免因格式偏差导致程序警告或界面不刷新，确保解析数据可稳定进入展示层。数据解析操作效果如图 5.7 所示。

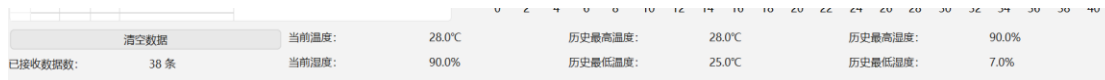


图 5.6 数据接收操作效果

图表实时刷新调试: 连续发送不同温湿度数据, 验证图表曲线是否实时增长、折线显示连贯、坐标刻度适配稳定。此步骤用于确认绘图模块重绘触发机制与数据映射逻辑正常, 确保曲线无断裂或绘制异常, 为系统趋势分析提供直观且稳定的可视化输出。图表刷新效果如图 5.8 所示。

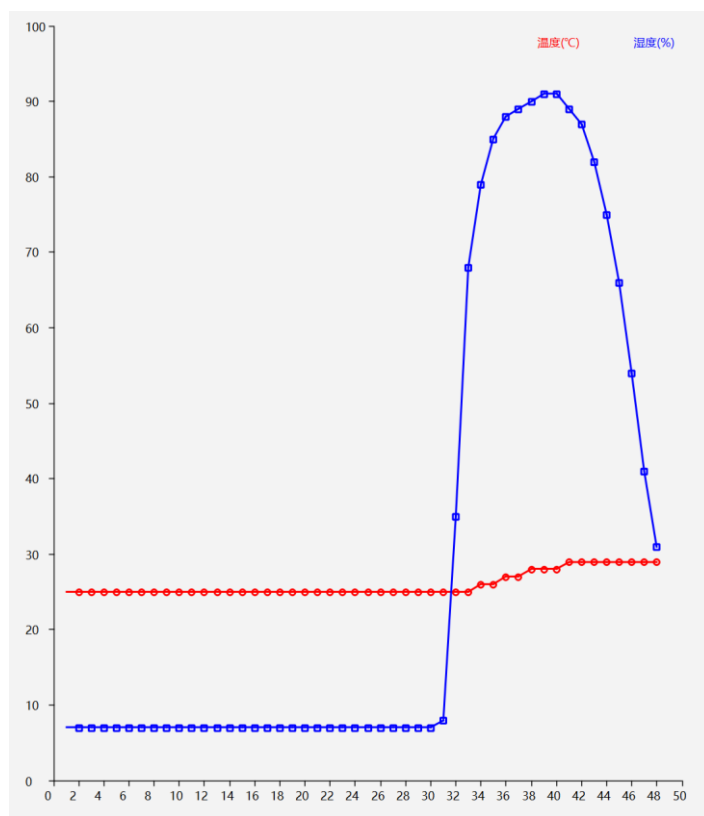


图 5.6 图表刷新操作效果

6 心得体会

实习期间，我严格遵循企业级嵌入式与串口可视化监控项目的开发流程，完成了从需求分析、固件开发、硬件联调到上位机可视化的完整数据闭环。

在需求分析阶段，我与指导工程师共同明确系统目标：稳定采集 DHT11 温湿度数据、UART 持续上报、Qt 端实时解析并可视化展示与极值统计，并统一了数据帧格式与串口通信参数（波特率、字段顺序、上报节奏、通信周期），确保固件与上位机解析逻辑一致可靠，为后续开发提供清晰依据。

在系统设计阶段，我严格遵循企业编码规范完成模块划分与接口定义，使传感器读取、数据打包、串口发送、回调接收与图表重绘等流程职责清晰、逻辑边界明确、资源占用可控、结构统一，保障系统稳定性与可维护性。

开发阶段中，我实现了串口端口枚举、结构化数据帧接收与解析、折线图实时绘制、温湿度极值更新与界面重绘触发等核心功能，完成了硬件采集 → UART 传输 → Qt 解析 → GUI 显示的全链路闭环监控逻辑。

在固件验证阶段，我使用 Keil 完成 STM32 在线烧录与运行调试，通过观察 LED 心跳稳定翻转、串口持续输出结构化数据帧、主循环无阻塞或异常中断，确认硬件电路与固件联调稳定可靠，系统成功进入主循环并持续采集与上报数据。

上位机联调过程中，我重点完成系统优化，包括：限制折线图数据点数量、优化串口缓存读取策略、规范管理串口资源占用与释放、增加异常识别机制，确保系统在长时间运行下无资源冲突、无阻塞、无数据丢失或乱码，提升监控可靠性与工程可维护性，使我深入理解了企业工程中通信容错、时序管理与数据一致性保障的重要性。

实习期间，我严格遵守团队纪律，做到准时到岗、规范使用开发工具、按时参会、及时汇报联调进展，保持主动推进意识，有效体现了工程执行中的严谨性与职业责任感。

总体而言，本次实习不仅完成了项目任务，也提升了我在嵌入式数据采集、通信上报、硬件联调与可视化优化方面的工程实践能力，为未来从事工业物联网采集与实时可视化监控方向奠定了坚实基础。